Open Access | https://doi.org/10.25080/hwcj9957



SciPy 2025

July 7 - July 13, 2025

Proceedings of the 24th Python in Science Conference
ISSN: 2575-9752

Jupyter Book 2 and the MyST Document Stack

A modular, extensible, web-native stack for authoring and publishing computational narratives

Project Jupyter[1], **Evan Bolyen**[2], **J Gregory Caporaso**[1,2], **Rowan Cockett**[1,3,4], **Danny Garside**[5,6], **Chris Holdgraf**[7,1], **Angus Hollands**[7,1], **Julia Kent**[8], **Franklin Koch**[3,1], **Jim Madge**[9,5], **Matthew McKay**[10], **Mike Morrison**[3], **Fernando Pérez**[11,12,13,14,1], **Steve Purves**[1,3,4], **Min Ragan Kelley**[1], **Brian E. J. Rose**[15], **Malvika Sharan**[5], **Brigitta M. Sipőcz**[16,5,17], **Stéfan J. van der Walt**[12,17], and **Kirstie Jane Whitaker**[12,1,5]

[1]Project Jupyter, [2]Northern Arizona University, [3]Curvenote Inc., [4]Continuous Science Foundation, [5]The Turing Way, [6]Digital Research Academy, [7]2i2c, [8]NSF National Center for Atmospheric Research, [9]Alan Turing Institute, [10]QuantEcon, [11]Department of Statistics, UC Berkeley, [12]Berkeley Institute for Data Science (BIDS), [13]Eric and Wendy Schmidt Data Science & Environment Center (DSE), UC Berkeley, [14]Data Science and Technology, Lawrence Berkeley National Lab, [15]University at Albany (State University of New York), [16]Caltech/IPAC - NASA/IPAC Infrared Science Archive, [17]Scientific Python

Abstract

Jupyter Book allows researchers and educators to create books, articles, and collections of connected documents that are reusable, reproducible, and interactive. Over the past three years, Jupyter Book has been entirely rebuilt on top of the MyST Document Engine and its ecosystem of tools and standards. This new foundation introduces a scalable way to publish modern scientific content that enables interactive code and computation, structured metadata, and content reuse across contexts. It allows for a more fluid workflow between computational discovery and communication, treating computation as *first-class* in both authoring and reading. Jupyter Book 2 (JB2) is now a Jupyter sub-project, and already powers several open scientific resources including The Turing Way, QuantEcon, Project Pythia, and the QIIME 2 Framework (Q2F) documentation ecosystem. This paper introduces the principles, architecture, and capabilities of JB2, shares real-world adoption stories, and explores how these tools are shaping the future of open computational publishing.

Keywords open science, jupyter, scientific communication, publishing, authoring, reproducibility

Published Jul 10, 2025

Correspondence to
Rowan Cockett
rowan@curvenote.com

Open Access

Copyright © 2025 Jupyter *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Jupyter Book [1] has become a widely used tool in computational science and open publishing, powering over 18,000 publicly available books[1], lectures, tutorials, and knowledge repositories. It enables clear communication of both exploratory computation and narrative text, making it possible to document, share, and publish computational workflows in accessible formats.

With Jupyter Book 2 (JB2), the project has undergone a major transformation. Jupyter Book

[1]There are 18,152 dependencies of JupyterBook (v1 and v2, combined) based on GitHub dependency graph: https://github.com/jupyter-book/jupyter-book/network/dependents.

2 is now a distribution of the MyST (Markedly Structured Text) Document Engine[2]; every book built with Jupyter Book 2 is also a MyST project, and building a book with Jupyter Book 2 means building a MyST project. Jupyter Book 2 heavily utilizes components from the MyST ecosystem—a collection of open source tools and standards for authoring open computational narratives. This is similar to the relationship between Jupyter Book 1 and Sphinx.

The goal of this overhaul is to make structured scientific content more interoperable, composable, extensible, and machine-readable. This enables researchers to publish high-quality, interactive content with support for standards-based reuse, full-text APIs, executable notebooks, and output formats like Websites, Typst, PDF, LaTeX, Microsoft Word, JATS XML (Journal Article Tag Suite, used in scientific publishing), and more. It is a major step forward towards Jupyter's vision of thinking and storytelling with code and data [2], [3].

In this article, we: (a) document the guiding principles and design thinking behind this major release; (b) present the high-level architecture behind Jupyter Book and the MyST ecosystem; (c) compare our tools to others in the scientific publishing ecosystem; and (d) highlight case studies of notable projects.

2. The challenge: Bridging Exploration and Publication

The mission of Jupyter Book and MyST is to build composable tools and standards for communicating technical and computational narratives. This must encompass the *full lifecycle of discovery and communication* from informal lab notes and collaborative tutorials to formal research articles and textbooks. Our goal is to support authors, educators, and researchers as they share work within local communities and publish across the global scientific commons.

We are focused on two overlapping but distinct communities:

Data-driven communicators Researchers, analysts, and educators who need to share working, reproducible content with collaborators, students, or the public. These users rely on Jupyter, Python, and other computational tools in their daily workflows, but often lack well-integrated systems to communicate what they've done or how to reproduce it.

Scholarly Authors and Publishers Scientists and academics preparing content for peer review, journals, repositories, or formal documentation, and organizations that serve these users. They need structured metadata, citation systems, and outputs compatible with academic and institutional publishing pipelines.

Both of these groups are underserved by the current ecosystem of open source technology. Today's scientific workflows rely on computational tools like Jupyter Notebooks, yet the systems used to communicate and publish these workflows are fragmented and poorly integrated. For example, researchers often do their work in an interactive computing environment designed for exploration, and must then adopt an entirely new toolchain for publishing their work. As a result, researchers must spend time adapting to new workflows, duplicating and reformatting their content just for the sake of publishing. Because output-specific pipelines, such as PDF generation, are fairly constrained, published work is often stripped of its computational context, leaving no mechanism to access software environments and reproduce results.

Over the years, there have been many attempts at improving this workflow with new technologies and standards (several are discussed below), but none have provided the combination of flexibility, standards-based implementation, and community governance

[2]MyST is pronounced like "missed" (see the docs).

needed to cover the entire lifecycle from idea to communication described above. Jupyter Book 2 and the MyST Ecosystem is an attempt to bridge this gap.

2.1. Overview of Jupyter Book 2's architecture

This section describes the architectural structure and components that drive the Jupyter Book 2 stack and the MyST ecosystem. At the heart of JB2 and the MyST ecosystem are four pieces:

- **The MyST Document Engine** (https://mystmd.org), a publication system for authoring computational narratives.
- **The MyST Specification** (https://mystmd.org/spec), a structured and semantically-meaningful way of representing computational content in an Abstract Syntax Tree (AST).
- **MyST Renderers and Themes** (https://mystmd.org/guide/overview#overview-themes), transform MyST ASTs into end-user outputs (websites, PDFs, etc.).
- **MyST Markdown** A lightweight extension of CommonMark Markdown[3] and GiHub Flavored Markdown (GFM[4]) which adds roles and directives (described below).

JB2 combines these and other MyST components into an end-user application meant for community knowledge bases and books. This new architecture is designed to enable dynamic rendering, multi-format output, and composable content workflows—backed by a structured document model that makes metadata, structure, and computation portable and machine-readable.

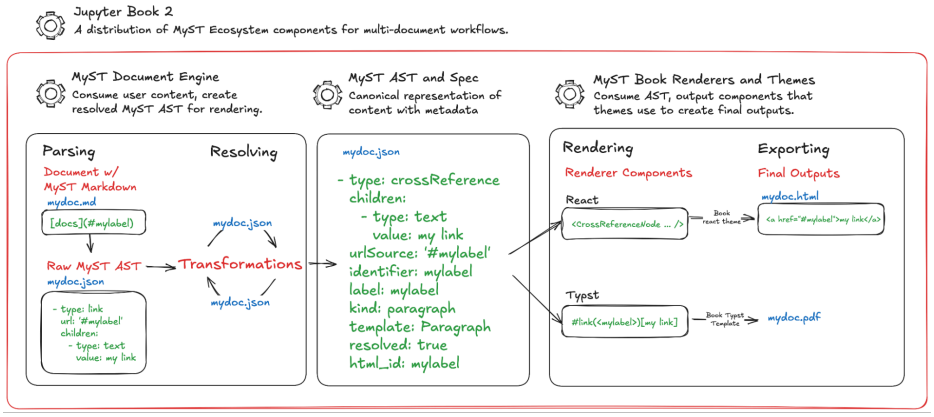


Figure 1. ***High-level overview of Jupyter Book 2 components.*** *JB2 combines several components in the MyST ecosystem, configured for multi-document workflows such as community knowledge bases, websites, and books. The MyST Document Engine parses MyST Markdown (and other flavors of text-based content) into an Abstract Syntax Tree (AST) that follows the MyST Document Specification. It resolves and transforms this AST to enrich it with extra document metadata (such as resolving DOIs). The engine then renders this AST to React components that are bundled with the Book Theme, which makes up the end-user application most-often used with JB2.*

2.2. The MyST Engine: A simple, powerful, and extensible document engine

The MyST Document Engine allows users to **parse** documents into a structured format, **execute** computational content, **transform** them to include metadata and resolve references, and **output** a resolved MyST Abstract Syntax Tree (AST, described in more detail below). It is run via the command-line and is written in TypeScript. It parses many types

[3] CommonMark (https://commonmark.org) is an unambiguous specification for the Markdown language with a formatting test suite.

[4] GitHub Flavored Markdown (https://github.github.com/gfm/), often shortened as GFM, is the dialect of Markdown that is supported for user content on GitHub.

of inputs—Markdown files, Jupyter Notebooks, LaTeX documents, external MyST projects, configuration files, etc—and ensures that all of their content is embedded in this output AST.

2.3. The MyST Specification: A Structured Computational Document Format

The **MyST Specification** defines all of the types of content and metadata that make up a computational narrative. It is meant to be a cross-platform standard that can be used by various applications and parsers in order to facilitate sharing and re-use of content.

The MyST Document Engine outputs a **MyST Abstract Syntax Tree (AST)** that follows the MyST Specification. The AST is a machine-readable representation of each document's content and structure. It preserves not only the narrative content, but also computational blocks, computational outputs from Jupyter Notebooks, citations, figures, and metadata such as author affiliations and license information. These are all built into a structured, versioned AST. It is a **Canonical Document Representation** that can be parsed and rendered into user-facing outputs by renderers and themes.

MyST Markdown is a flavor of Markdown written for the MyST Specification that gives authors quick access to these and other extension points. MyST Markdown supports syntax for all of the content types supported in the MyST Specification. It is used both in Jupyter Book 2 and the MyST Engine, as well as in the Sphinx ecosystem via the myst-parser extension.

As an example, see the example **note directive** written in MyST Markdown below (for more information on directives, see the sections below).

```
:::{note} Admonition title
Admonition **body**.
:::
```

The MyST Document Engine parses this Markdown content into an AST that follows the MyST Specification. It looks like this:

```
- type: admonition
  kind: note
  children:
    - type: admonitionTitle
      children:
        - type: text
          value: Admonition title
    - type: paragraph
      children:
        - type: text
          value: 'Admonition '
        - type: strong
          children:
            - type: text
              value: body
        - type: text
          value: ...
```

Renderers then convert this AST into various output formats.

2.4. MyST Renderers and Themes: Tools to convert MyST AST into user-facing products

MyST Renderers and Themes are tools for converting documents that follow the MyST Specification into user-facing outputs, such as websites and PDFs. Renderers are essentially a collection of rules for converting each type of content (called a "node" in a MyST AST) into

an output. For example, the MyST AST for the Note admonition above would be rendered into a React component, and exported to HTML via the static HTML export command `jupyter book build --html` like so:

```
<-- React component used by the Book or Article theme -->
<Admonition title="Note" kind="{AdmonitionKind.note}" color="blue">
  This is an admonition!
</Admonition>

<-- React component converted to this HTML after static HTML export -->
<aside className="myst-admonition myst-admonition-note">
  <div className="myst-admonition-header">
    <InformationCircleIcon className="myst-admonition-header-icon" />
    <div className="myst-admonition-header-text">Note</div>
  </div>
  <div className="myst-admonition-body">This is an admonition!</div>
</aside>
```

The **Book React Theme** defines the basic structure and styles of a Jupyter Book 2, and is the default result that most users get when they render their books as websites. The React component above is styled by the Book Theme into this final product:

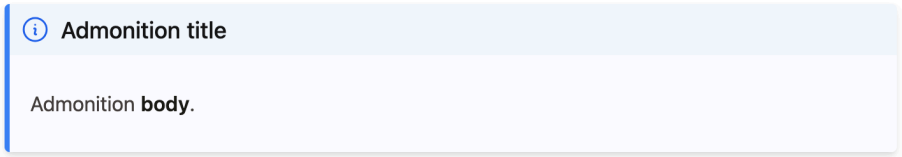


Figure 2. ***Example admonition component.*** *The final product of a MyST Admonition rendered by the Book Theme. Arguments and keywords in the `{note}` directive influence the look and feel of this final product.*

Importantly, **MyST Renderers can be written for any kind of output**. For example, there is a **Typst** renderer and a **LaTeX** renderer, which can themselves be customized with **themes** to generate different types of outputs. Jupyter Book 2 ships with a default Book Theme for each type of renderer, but we imagine our ecosystem of users extending this to include, for example, LaTeX templates for a variety of Journals. The `myst-templates` GitHub organization (github.com/myst-templates) already hosts a collection of community-maintained templates for rendering and theming MyST AST into various kinds of outputs.

2.5. Jupyter Book 2: A distribution of MyST components

Finally, Jupyter Book 2 is a lightweight distribution and configuration layer on top of these and other components of the MyST ecosystem. It is meant to provide an opinionated experience of MyST–for example, by turning on certain features by default, and by providing an out-of-the-box theme meant for multi-document websites. At its core, Jupyter Book 2 functionality is leveraging components in the MyST ecosystem.

This is a pattern that we wish to encourage and grow in the MyST ecosystem. Tools like the MyST Document Engine aim to be unopinionated, modular, configurable, and extendable. They're primarily meant for use by **power users and developers** who are interested in configuring MyST tools for their own needs/community, or building MyST tools into separate applications. Jupyter Book 2 is meant for **end-users like researchers and educators** who want an opinionated system for communicating computational narratives that "just works".

We envision that a broader *ecosystem* of high-level tools and services—*similar to Jupyter Book 2* itself—may be built on top of the MyST Document Engine, Specification, Renderers,

and other components. With that in mind, we now turn to the design principles that drove the architecture of the MyST ecosystem, and the value that they bring to the open science community.

3. Design Principles

Jupyter Book 2 is designed to meet the practical needs of researchers, educators, and publishers who communicate with—and about—data. It also supports developers in the open science ecosystem who build applications on shared standards and tools. We have grounded the design and development of the JB2 stack with the following principles.

Simple to use, easy to extend Users should be able to run common workflows with minimal complexity, dependencies, setup, and configuration. Power-users should be able to extend this functionality with minimal effort.

Use standardized, machine readable formats There should be a canonical, machine-readable representation of all content, allowing it to be re-used and rendered into multiple outputs without losing structure, metadata, or intent.

Content should be modular and composable Canonical content should be made up of small, reusable components to enable reuse, cross-referencing, and re-mixing into diverse outputs.

Computation should be first-class Code, data, and outputs are treated as integral components of the research narrative, for example, integrated and reproducible ways to calculate figures, show inline values, and cache computations.

Open science infrastructure should be governed by an open community Jupyter Book and MyST should be stakeholder-led, with a diverse community of organizations and people from the open science community that guide its development.

Below, we describe each in more detail, how it is implemented in the MyST ecosystem, and the value that it brings to communicating open science with computational narratives.

3.1. Simple and quick to use, easy to extend

First and foremost, it should be simple and straightforward for authors to create and publish computational narratives with Jupyter Book 2, requiring little configuration and customization. If the out-of-the-box functionality of Jupyter Book 2 doesn't fit an author's needs, there should also be simple mechanisms to *extend* Jupyter Book's functionality. The MyST ecosystem should be composed of independent pieces that can work together as a whole, with natural ways to extend for new functionality.

3.1.1. *MyST's plugin ecosystem*

The MyST Documentation engine has several key **extension points** that allow users to add new functionality in a controlled way[5]. Below are a few key extension points:

Roles and directives are mechanisms for adding new types of in-line and block-level content (respectively), allowing developers to define content abstractions (similar to functions in a programming language) for use by authors. Users may define their own roles and directives using JavaScript, Python, or any other language that reads and writes from `stdout`. Building a new role or directive involves taking MyST AST as input, applying transformations, and returning AST.

[5]Examples are available in the appendix

Transforms are a mechanism for transforming the content of one or more MyST ASTs. It is used to enrich content as part of the build process. For example, by collecting `{figure}` nodes throughout the page and aggregating them into a single gallery.

Renderers are a way to export MyST AST into arbitrary types of outputs. They're useful for converting the "Canonical document" into many different formats or styles without changing the original content. They are explained in sections below.

3.2. Content should be canonical and machine readable

The MyST ecosystem emphasizes a **"content-first" model**, where authoring is separated from output format. This reduces the friction and wasted time associated with re-packaging source content in order to build a slightly different output.

3.2.1. *The MyST AST is semantically structured*

The MyST Document Engine produces a machine-readable abstract syntax tree for every page. This well-structured build artifact is a canonical document that describes the underlying structure, content, and metadata of the document. This internal representation captures not just the text of a document, but also its semantic components: citations, headings, roles, directives, code blocks, figures, equations, and embedded metadata like Open Researcher and Contributor IDs (ORCIDs), Digital Object Identifiers (DOIs), or author affiliations via the Research Organization Registry (ROR). A canonical version also allows content to be rendered into multiple outputs—without losing structure, metadata, or intent.

By embedding all of the relevant metadata in a MyST Document, it is possible to gain a more rich understanding of the intent, context, and learning contained within a document. It is also possible to (re)use subsets of this information for various end purposes (for example, to re-render a figure, or to provide metadata for a scientific discovery engine). We give specific effort to support the open-science ecosystem, especially around structured, scholarly metadata that help discoverability (e.g., DOIs, ORCIDs, RORs).

3.2.2. *Many renderers operate on the same AST*

The AST serves as the canonical version for rendering content into multiple target formats: web-native HTML, JATS XML, PDF via Typst or LaTeX, Microsoft Word, and more. Exporting to a target format is done with a **renderer** that consumes the AST and applies format-specific styling, layout, and transformations. Our attention is primarily focused on the React-renderers, Typst PDF templates (a new PDF renderer that is significantly faster than LaTeX), and JATS XML used in scholarly publishing. For example, the SciPy Proceedings for 2024 and 2025 both use the React renderers, the Typst renderer for PDF creation, and JATS XML for all articles [4].

By treating that document AST as a first-class build output, authors can write their content once, and publish in multiple places without manually adapting their content for each target format. For example, a MyST Document can be built once, and the resulting AST can be rendered into an HTML website as well as multiple LaTeX documents for submission to various journals. Paired with the machine readability of the AST (the `.json` file served with websites), it is possible to render multiple "views" of the same underlying content without creating confusion for where the source of truth lies.

Additionally, having a standard and metadata-rich AST structure allows many different workflows to be served from the same document engine. For example, while this article focuses on the multi-document workflow of Jupyter Book 2, the MyST Document Engine has also been used as part of *scholarly article publishing pipelines*, which are composed of independent articles that benefit from the same ability to cross-reference one another.

For example the Proceedings of SciPy have been published with the MyST Document Engine for the last two years, the Notebooks Now! project [5] incorporated MyST-based publishing into the American Geophysical Union proceedings for the last several years, the NeuroLibre project has built a pre-print server using the MyST engine, and Curvenote builds a publishing platform using MyST components [4]. By enabling a variety of publication output types with the same underlying document engine, the MyST ecosystem can reduce duplicated technology and streamline the workflow from data science and documentation to publishing.

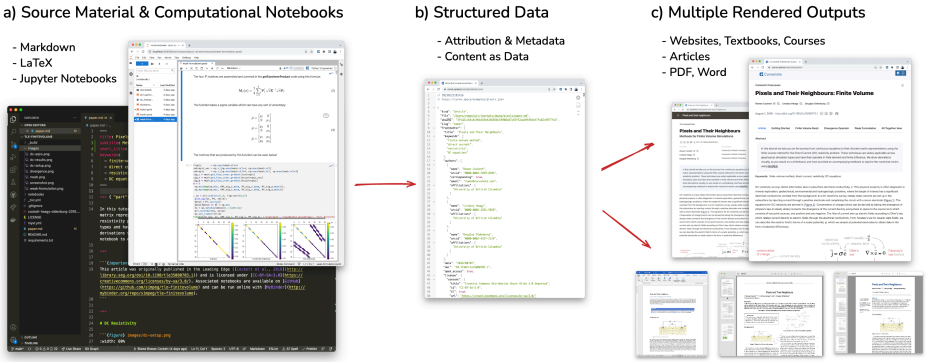


Figure 3. ***Rendering a single AST into many output types.*** *In this workflow, a single collection of source files (a) are built into a MyST AST (b) that serves as the canonical representation of the document. One or more renderers operate on that AST and convert them into outputs (c) such as HTML, PDF, Microsoft Word, etc. (Figure adapted from R. Cockett, S. Purves, F. Koch, and M. Morrison [4], showing content from R. Cockett, L. J. Heagy, and D. W. Oldenburg [6]; CC-BY-SA-4.0).*

3.3. Content should be modular and composable

The MyST ecosystem is designed for composition. Documents are structured into small, addressable components: sections, figures, equations, terms, figures, outputs—that can be reused across books, projects, and publications. This modularity supports new patterns of collaboration and publication: a figure generated in a course textbook can be cited and embedded in a tutorial or a journal article without duplication or loss of provenance. A community policy or database can be re-used throughout many pages in its documentation without duplicating information and creating multiple sources of truth. Additionally, configuration for a collection of projects can be shared and reused.

3.3.1. *The MyST AST modularizes content for easy parsing and re-mixing*

Each page or resource in a Jupyter Book project is assigned a unique slug (a human-readable URL) and corresponding AST. These can be referenced across projects, embedded in-line with other page content, or previewed when users hover over links. This is based on both the MyST Specification and a machine-readable cross-reference manifest `myst.xref.json`[6]. Similar to intersphinx references, a feature for cross-project linking in the Sphinx documentation generator, the `myst.xref.json` gives stable links to any aspect of the content. This makes it easy to programmatically learn the structure of a Jupyter Book, referenceable labels, and the location of the canonical content for each document.

An author can then reference labels in a different Jupyter Book or MyST project with an `xref` link. For example, the MyST Markdown below demonstrates referencing an external project called `other-project` (assuming that the URL of the other project is defined in your MyST configuration).

[6] An example is available in the appendix

```
[Link text](xref:other-project#label-name)
```

This also allows authors to reference *any other Jupyter Book or MyST project* , and use `xref:` links to link or embed the content. These links are either generated at build- or render-time, and allow for dynamic linking between projects with a simple user interface. Paired with structured JSON APIs for document content (described below), it also allows machines to efficiently access data in a Jupyter Book (for example, to enable hover-previews or to identify source files for HTML pages for use by Large Language Models)[7].

3.3.2. *API-Accessible and Federated by Design*

The Canonical AST of each document is *designed to be re-used by others.* It is packaged as a machine-readable `.json` file to encourage programmatic re-use. It is also bundled with web outputs to encourage external services or readers to access and utilize the AST via a machine-readable metadata endpoint (`/pagename.json`).

Machine readable content accessible via an API is one of the most transformative aspects of the JB2 architecture. By making each canonical document accessible via a `.json` API endpoint that exposes its AST and metadata, content can be programmatically indexed, referenced, embedded, transcluded, or transformed without needing to scrape or recompile the site. Readers and developers can query structure, citations, and content across books, enabling new tools like federated search, content previews, and live citation graphs. It also enhances the reading experience by enabling full-text search, granular cross-referencing, and live content embedding across projects.

Moreover, by aligning with community standards (e.g., JATS, DOIs, ORCID, ROR), the system supports today's scholarly publishing workflows, remains grounded in lightweight, Markdown-based authoring, and opens up possibilities for more composable workflows in the future for scientific authoring [7]. This approach lays the groundwork for a **federated publishing network** where content from different lab groups, institutions, or journals can be referenced, reused, and remixed while maintaining proper attribution and structure.

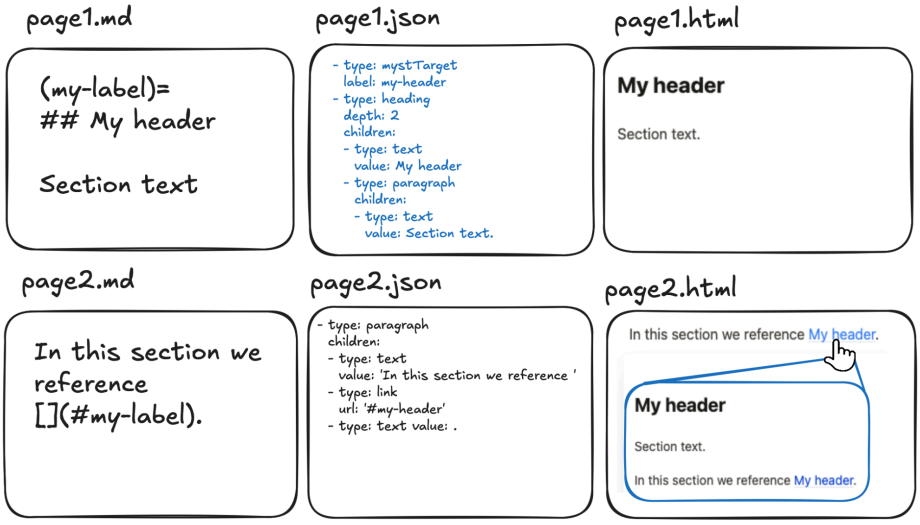


Figure 4. ***Example of how machine-readable AST enables cross-page and cross-project hover references.*** *In this abstract example we demonstrate how the source content of a page (`page1.md`) is parsed into an AST (`page1.json`) with a unique label called `my-label`. A second page (`page2.md`) references this label.*

[7]The hover-xref project in sphinx was an attempt at doing this

At rendering time (`page2.html`) the content of the label is pulled into the page and displayed via a hover-pop-up.

3.3.3. *The MyST Specification is a versioned, community-governed standard*

Just as the Jupyter Notebook format (`.ipynb`) has enabled interoperability across dozens of tools by serving as a stable, well-defined format, the MyST AST aims to serve a similar role for scientific communication. The AST specification is versioned and documented, and efforts are underway to standardize it further via the `myst-spec` initiative. We are also investing in forward-and-backwards interoperability in this specification in the MyST Document Engine.

By creating a community process for defining the evolution of the MyST Document and Markdown standards, we aim to create a participatory process to ensure that this ecosystem continues to serve the needs of its key stakeholders: members of open science, open source, and open knowledge communities who are passionate about communicating with computational narratives. Coupled with the dependability of a stable specification, and a technology stack that allows for graceful upgrading and downgrading, this allows developers, publishers, and downstream platforms to build confidently on top of the MyST ecosystem.

3.4. Computation should be first-class

A key challenge in communicating with data lies in the fact that tools and systems for communicating are *separate* from tools for making discoveries with data and computation. Many researchers begin their communication journey in a computational notebook—exploring data, testing hypotheses, or sharing early-stage insights. Bringing that work into a readable, shareable, and publishable format traditionally requires significant rework: copying plots into a document, cleaning outputs, manually formatting results, re-structuring to suit publication format requirements. These extra steps not only introduce friction, but also weaken reproducibility and create a disconnect between the computation and its communication.

3.4.1. *Execution and caching are built into the MyST Document Engine*

MyST comes with infrastructure to parse computational content, execute it[8], cache the results, and insert them into the MyST AST so that they may be used and referenced. Computational content can come in two forms:

Jupyter Notebooks (`.ipynb` files) This is the standard structure for Jupyter Notebooks, and the MyST Document Engine will use the Jupyter kernel metadata in the notebook to execute each cell and embed its content in the output.

MyST Markdown Notebooks (`.md` files) The MyST Engine allows authors to define the structure of a Jupyter Notebook with a Markdown file, using a special directive called `{code-cell}`. Paired with text-to-notebook conversion tools like Jupytext, this allows for two-directional conversion between `.ipynb` files and MyST Markdown Notebooks.

3.4.2. *Computational outputs are treated like any other content*

Aside from simply executing and inserting outputs into a MyST document, these outputs can also be used similar to any other type of MyST content. For example, authors can **attach a label to a notebook cell** in order to **reference its outputs** like they would any other kind

[8]The Jupyter Book team intends to extend computational capabilities to use kernels and execution services that are hosted elsewhere. This will make it possible for the computation in a Jupyter Book to be executed in a remote environment that may have access to more complex resources or sensitive data (for example, as part of a journal publication pipeline for reproducibility).

of content. For example, the `{code-cell}` directive below creates a plot with `matplotlib` in `computational-page.md`:

```
(full-plot-context)=
This code generates an image:

```{code-cell} python
:label: my-plot
plt.scatter([1, 2, 3], [4, 5, 6])
```
```

In `another-page.md`, the author is able to insert this output into a figure with a caption like so:

```
Below you see the results of my computation:

:::{figure} #my-plot
Here's a caption for the image referenced by `my-plot`, see [the full plot context](#full-plot-context) for details.
:::
```

Our goal with Jupyter Book 2 is to integrate computational content at every level. Code, figures, outputs, and interactive elements are embedded directly in the narrative and treated as primary content—not sidebars or supplemental material. This increases the likelihood that authors will interweave computational content into the reading experience, increases the reproducibility and re-usability of computational ideas, and allows for more powerful storytelling through computation, data, and software [2].

3.4.3. *In-page computation brings computational interactivity to the reading experience*

Beyond static computational outputs, JB2 embraces the web-native workflows of modern data science by allowing computation to power interactive experiences with a live kernel. Executable content can be powered by any Jupyter Kernel by leveraging MyST's native integration with Thebe, which enables static documents to become interactive by linking code cells with a computational kernel. Thebe can leverage kernel providers from sources like Binder [8], JupyterHub, or a local Jupyter Server, allowing readers to rerun computations on demand, right from the page without leaving the context of the narrative.

Providing computational interactivity without leaving the page drastically reduces the friction required to interact with computational ideas. It gives authors the ability to power their computational content with arbitrary software, computational resources, or access to data (for example, authors could leverage a service that operates a BinderHub that provides access to GPUs and a 10 Terabyte dataset, and provide interactive kernels for their readers that are powered by that hub). Moreover, modern efforts to package computational environments in web-native toolchains like WebAssembly (via tools like JupyterLite) will allow for interactive execution directly in a reader's browser.

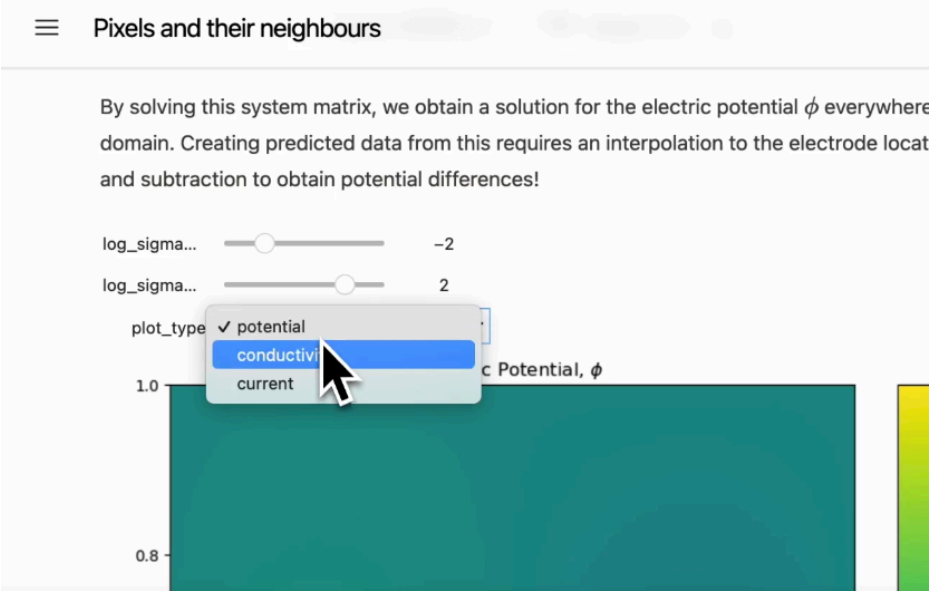


Figure 5. *Embedded notebook cells with live computation directly in an articles with computation backed by Jupyter. These can be running on BinderHub or directly in your browser through JupyterLite. Originally published in R. Cockett, S. Purves, F. Koch, and M. Morrison [4].*

3.5. Open science infrastructure should be governed by an open community

Finally, we believe it is crucial for open science technology and standards to be governed and led by a community of stakeholders, not driven by a single person or organization. Jupyter Book and MyST are community-led projects, developed and governed by an open community of contributors, and hosted under the umbrella of Project Jupyter. The tools in the Jupyter Book 2 stack are maintained by a contributor community that includes open infrastructure organizations, universities, independent developers, and end-users who rely on them in practice.

To facilitate participatory and collaborative design as the Jupyter Book 2 ecosystem evolves, all team practices and policies are defined in the Jupyter Book Team Compass, which inherits its open governance and practices from Project Jupyter. The contributing guide provides detail of the overall stack to help others understand and participate in the development. The MyST Enhancement Proposal process facilitates community feedback to evolve the MyST Specification and Markdown language. These practices aim to allow the MyST ecosystem to remain a community-supported standard while continuing to evolve to meet the needs of its users.

3.6. Next directions for Jupyter Book 2

This article describes the high level architecture, goals, and guiding principles for the JupyterBook project; there is still much work to be done to fully realize our ambitions. We have tried to make it clear where our goals are still aspirational, and we invite others to join our community as we develop these ideas further.

There are several directions where we are particularly excited to make improvements.

We aim to lean further into the idea of modular and composable content by **providing directives and roles that create summaries and views of MyST content**. This could be used to create summary content like galleries and blogs with local or remote MyST documents and Jupyter Books. We wish to **add support for API documentation to enable software development documentation**. We aim to improve **integration of computational work-**

flows with external services and infrastructure such as JupyterHub or Binder to allow authors to leverage extra resources or sources of data as part of generating their books. Finally, we aim to **enable more extension points in the underlying MyST Engine** to give authors and developers more control over the outputs, look and feel, and new kinds of functionality that MyST can support.

4. Comparison to Other Tools

Jupyter Book 2 sits among a number of tools for authoring and publishing computational and scientific content. These include static site generators, document converters, and academic publishing frameworks. Many of these systems are well-suited to particular workflows —whether technical documentation, reproducible research, or educational materials—but often focus on specific output formats, assume linear document structures, or lack integration with browser-based and computational environments. Moreover, most document engines are produced by a single person or organization, rather than a multi-stakeholder community. JB2 is designed to address these gaps, particularly for communities working in open science.

Jupyter Book 1, developed by the Executable Books Project, is built on Sphinx, a Python documentation generator that enabled integration with reStructuredText, MyST extensions, and a range of HTML output themes. While powerful and extensible, Sphinx was originally designed as a documentation engine for software projects. Support for scientific markup and computation has been added by the community, extending its original design. However, its design principles and architecture are not optimized for the browser-based authoring, computation, content re-use, and efficient multi-output exporting workflows that are core to the JB2 stack.

Jupyter Notebooks, developed by Project Jupyter, is a document format and web-based interface for authoring computational narratives with data. Jupyter Notebooks have support for interactive outputs and computation that is similar to Jupyter Book. However, they focus on individual documents rather than multi-page projects with references between them. They are also designed for data science and exploration rather than communicating more complex and re-usable computational narratives. Jupyter Book 2 *complements* the Jupyter Notebook by providing a more rich feature set and workflow that focuses on authoring and reading documents. A Jupyter Book can be written as a collection of *Jupyter Notebooks*, and knows how to parse MyST Markdown in notebook cells. Jupyter Notebooks and Jupyter Lab can also incorporate some of Jupyter Book 2's functionality into their interfaces via the `jupyterlab-myst` extension.

Quarto, developed by Posit PBC, is a flexible publishing system supporting Jupyter notebooks, Markdown, and multiple output formats including HTML, PDF, DOCX, and JATS. Quarto is built on top of Pandoc and is tightly integrated with the R, RStudio, and Positron ecosystems. At this time, Quarto is more polished and further in its development lifecycle, with better support for features like PDF rendering (e.g., sub-figures and tables), and presentation output formats. While Quarto has excellent support for static HTML outputs, it does not focus on the principles of modularity and composability which are core to Jupyter Book 2 (e.g., it does not have content APIs and structured build outputs meant for reuse, nor expose packages that can be used by other tools). Quarto is built directly on Pandoc, which uses a GPLv2 license; the Quarto command line tool is MIT licensed since version 1.4, however, many of the core packages, extensions, and utilities are not permissively licensed. Finally, Quarto is developed by a single company, in contrast to Jupyter Book which is governed and developed by a multi-stakeholder community, and uses a permissive MIT license to maximize the re-use of its technology for individual and commercial purposes.

Other documentation and markup systems like Docusaurus, MkDocs, Hugo, quarkdown, typst, and djot are popular choices for technical documentation, developer blogs, PDF generation, and websites. These tools offer good performance, templating, and theming options, and in some cases also focus on extensibility. They are not designed for scientific publishing or computational narratives and they lack features such as executable content, math, and citation handling, semantic metadata, or integrating with persistent identifiers in the scholarly ecosystem (e.g., DOIs, RRIDs, or RORs) - all of which are built into the JB2 ecosystem.

While these tools each serve their respective audiences well, JB2 is focused specifically on the needs of computational and scientific communities. It supports both narrative and executable content, prioritizes open infrastructure and interoperability, and is designed to integrate with existing Jupyter workflows and the wider open-science ecosystems.

5. Case Studies

Jupyter Book 2 is already being used across a broad range of scientific and educational projects. From national training initiatives to domain-specific research platforms, these case studies demonstrate how JB2 supports reproducible publishing, modular content reuse, and scalable collaboration.

The Turing Way [9] is an open guide to reproducible, ethical, accessible, and inclusive data science. With hundreds of contributors and a highly active community, the project has adopted Jupyter Book 2 to manage its growing library of community-authored chapters and living documents. The structured document model and improved metadata handling in JB2 have made it easier for the Turing Way's community members to learn from, reference, and leverage one another's work. The move from Sphinx to MyST has greatly simplified the technology underlying The Turing Way, which makes it easier for new contributors to build the Turing Way locally and contribute to it. They have been able to reduce the complexity of their own infrastructure by implementing custom features as MyST plugins. This has simplified the book's deployment process and better allowed these features to be shared as modular, generic plugins rather than Turing-Way-specific scripts.

Project Pythia [10] is a community-driven effort to advance computational geoscience education, training, and knowledge-sharing through open-source learning materials. Pythia's collection of cross-referenced Jupyter Book sites draw from over 75 repositories and large numbers of contributors, making the structured content model and cross-repository referencing features of JB2 particularly valuable. Pythia recently transitioned to JB2. There were two standout features to the community: (1) the ability to compose and share configuration between sites, including things like footers, shared references, navigation links; and (2) the ability to cross-reference content between un-related projects. Both of these build on the composability and modularity designs that are at the core of JB2. As an inclusive open-geoscience community effort, Project Pythia aims to build tools and practices that reduce participation barriers and incentivize working geoscientists to share their discipline-specific data fluency. Shared configurations in JB2 substantially reduce the boilerplate code in individual repositories, clearing the way for contributors to focus on their science content while a core team of maintainers efficiently advances the shared computational and web-publishing infrastructure.

The QIIME 2 Framework (Q2F) is a platform for biological data science tools, originally developed for the QIIME 2 microbiome bioinformatics toolkit, but now supporting tools across diverse subdomains of biology including multiplexed serology and pathogen genomics. Q2F has adopted JB2 to support its documentation ecosystem which includes documentation that is general to users of all Q2F-based tools (e.g., creating and using a Q2F data artifact

cache) and documentation that is specific to subdomains (e.g., an end-to-end tutorial for microbiome data analysis). JB2 enables embedding of content across documentation projects, such that general content can be embedded in domain-specific content where relevant, making it easier to hide the software stack complexity from end users while avoiding duplication of source content.

QuantEcon provides open educational resources in economics and quantitative methods, built using Python, Julia, and Jupyter. The transition to JB2 included investment in a custom theme for JupyterBook, which allowed the project to modernize its publishing infrastructure while maintaining interactive notebooks and supporting export to multiple formats including HTML, PDF, and `.ipynb`. The upgraded theme allowed for adoption of new features like hover cross-reference between exercises and solutions using the proof and exercise extensions.

These case studies demonstrate the flexibility of JB2 to meet the needs of both education and research, large and small teams, and varied technical audiences. Each project benefits from the same core strengths: modular content, structured metadata, and tight integration with Jupyter.

6. Conclusion

Jupyter Book 2 is an open-source publishing system for computational and scientific content. It enables authors to combine Markdown, notebooks, structured metadata, and executable code into documents that can be rendered as websites, PDFs, JATS XML, Word documents, and more. By rebuilding the system around a structured document model and a web-native engine, MyST Document Engine, JB2 supports workflows that are modular, reproducible, and extensible. These qualities are essential for modern research and education, and bridge the gap between exploratory computation and formal, published communication, enabling authors to spend more time on data-driven discovery and communicating ideas, and less time re-purposing their workflows for publication.

Unlike general-purpose documentation systems, static website builders, or single-format converters, Jupyter Book 2 is *built by and for* researchers, educators, and institutions working with code, data, and narrative. Each page produced with JB2 has a structured, machine-readable representation that can be queried, embedded, or reused via an API. This enables a federated publishing model, where content from different projects, lab groups, or institutions can be referenced, previewed, and composed together—without duplication or loss of structure. The result is a more connected and discoverable ecosystem of scientific knowledge, where modular content can be maintained in place but surfaced in multiple contexts, with proper attribution and rich metadata preserved.

The Jupyter Book project is openly governed under Project Jupyter, actively maintained, and already powers many high-impact documentation and communication initiatives across domains. From reproducible data science guides like The Turing Way, to educational curricula like QuantEcon and Project Pythia, to domain-specific platforms like the QIIME 2 Framework, Jupyter Book 2 is helping authors publish richer, more reusable, and more transparent scientific outputs.

As scientific publishing continues to evolve, Jupyter Book offers a model for infrastructure that is interoperable, open by design, and responsive to the needs of its users. It is not just a tool for building books—it is a foundation for the next generation of computational publishing.

7. Appendix

7.1. History & Background

In this section, we share a brief history of the Jupyter Book project with the goal of providing context for the path that led to JB2, as well as to share credit and acknowledgement for the hundreds of contributors that have helped the project over the years.

JB2 is the third significant re-write of the Jupyter Book stack in the past decade, each of which involved hundreds of collaborators and users. The original version was built in 2018 as a loose collection of template files utilizing nbconvert for execution and Jekyll for rendering notebooks into websites, originally built for the Data 8 textbook.

In 2020, Jupyter Book was re-written and formally released as Jupyter Book 1 (JB1). This was built on the Sphinx documentation generator, and resulted in the creation of the MyST Markdown syntax and parser for Sphinx. This work was largely funded by the Executable Books Project, a funded by the Sloan Foundation (Grant #9231), Jupyter Meets the Earth (NSF grant #1928406). While the Executable Books Project has completed, maintenance of the Jupyter Book 1 stack continues to this day.

Between 2020 and 2023, the Executable Books Project began a parallel collaboration with the company Curvenote to explore a standards- and web-based workflow for the MyST Markup language (funded in part by Alberta Innovates and the Stanford Doerr School of Sustainability). As a result, Curvenote integrated their document engine as an upstream project in the `executablebooks/`organization and the project developed it together from there. This became the starting point for the `mystmd` stack described in this article.

From 2023 onward, the wider community has invested in `mystmd` and improved the capabilities of the command line tool, parsing capabilities, templates, and web themes. The next version of JupyterBook, Jupyter Book 2 (JB2), is built on top of this engine. In 2024, the project moved from being an independent organization to being incorporated as an official Jupyter sub-project, standardizing on using and stewarding the MyST document engine (See #123).

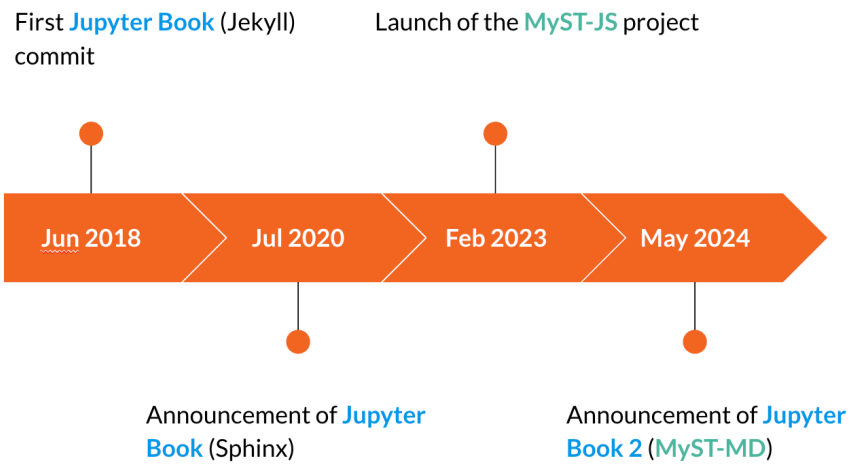


Figure 6. *Major phases of Jupyter Book development.*

7.1.1. *A note on migrating from Jupyter Book 1*

A key goal of Jupyter Book 2 was to leverage the design and standards from Jupyter Book 1 and the Sphinx stack in order to facilitate the upgrade process. As a result JB2 leverages

the same MyST Markdown syntax as JB1 with minimal disruption. It also aims to expose a key subset of the extension points that were available in JB1 and Sphinx (for example, roles, directives, custom transforms, etc). While there is a subset of functionality that is still unique to Sphinx, the JB2 team is focusing their efforts on developing key missing functionality to narrow this gap. See the Jupyter Book 2 migration guide for more information.

7.2. Examples of plugins

Directives: Here is an example of a directive logic that generates an `{image}` node in the AST by pulling a random image from picsum. Note how we define arguments and options, similar to how a function would be defined in a programming language.

```
const picsumDirective = {
  name: 'picsum',
  doc: 'An example directive for showing a nice random image at a custom size.',
  alias: ['random-pic'],
  arg: {
    type: String,
    doc: 'The ID of the image to use, e.g. 1',
  },
  options: {
    size: { type: String, doc: 'Size of the image, for example, `500x200`.' },
  },
  run(data) {
    // Parse size
    const match = (data.options?.size ?? '').match(/^(\d+)(?:x(\d+))?$/);
    let sizeQuery = '200/200';
    if (match) {
      const first = match[1];
      const second = match[2];
      sizeQuery = second ? `${first}/${second}` : first;
    }

    const idQuery = data.arg ? `id/${data.arg}/` : '';
    const url = `https://picsum.photos/${idQuery}${sizeQuery}`;
    const img = { type: 'image', url };
    return [img];
  },
};

const plugin = { name: 'Lorem Picsum Images', directives: [picsumDirective] };
export default plugin;
```

Transforms: Here is an example of transform logic that parses the MyST AST and replaces **strong** styling with *emphasis* styling. It operates on all `node` objects in the document using the `selectAll` utility function.

```
const plugin = {
  name: 'Strong to emphasis',
  transforms: [
    {
      name: 'transform-typography',
      doc: 'An example transform that rewrites bold text as text with emphasis.',
      stage: 'document',
      plugin: (_, utils) => (node) => {
        utils.selectAll('strong', node).forEach((strongNode) => {
          const childTextNodes = utils.selectAll('text', strongNode);
          const childText = childTextNodes.map((child) => child.value).join('');
          if (childText === 'special bold text') {
            strongNode['type'] = 'span';
            strongNode['style'] = {
              background: '-webkit-linear-gradient(20deg, #09009f, #E743D9)',
              '-webkit-background-clip': 'text',
              '-webkit-text-fill-color': 'transparent',
            };
          }
        });
      },
    },
  ],
};

export default plugin;
```

7.3. Example myst.xref.json

Here's an example of references in a `myst.xref.json` file from the MyST Guide, note both page references and "identifier" references (e.g., labels attached to a figure or header):

```
references:
{ kind: "page", data: "/index.json", url: "/" }
{ identifier: "cool-myst-features", kind: "heading", data: "/index.json", … }
{ identifier: "quickstart-tutorials", kind: "heading", data: "/index.json", … }
{ identifier: "cite-mystmd", kind: "heading", data: "/index.json", … }
{ identifier: "project-goals", kind: "heading", data: "/index.json", … }
{ kind: "page", data: "/installing.json", url: "/installing" }
{ identifier: "installing-myst-tabs", kind: "tabSet", data: "/installing.json", … }
```

7.4. Composable Configuration

MyST configuration files can be composed with one another using the `extends:` keyword. This allows configuration to be split across multiple files, or even downloaded from a remote source via the web. For example, an author might put their author affiliation information in a dedicated `authors.yml` file:

```yaml
version: 1
project:
  contributors:
    - id: person_a
      name: First Last
      email: person_a@org_a.org
      orcid: XXXX-XXXX-XXXX-XXXX
      github: person_a
      affiliations:
        - id: org_a
        - id: org_b
    - id: person_b
      name: First Last
      email: person_b@org_b.com
      orcid: XXXX-XXXX-XXXX-XXXX
      github: person_b
      affiliations:
        - id: org_b
  affiliations:
    - id: org_a
      name: Organization A Incorporated
    - id: org_b
      name: Org B Inc.
```

And then re-use this affiliation information across multiple Jupyter Books or MyST Projects by "extending" the configuration like so:

```yaml
version: 1
extends:
  # Our local authors file
  - authors.yml
  # A remote configuration file
  - https://raw.githubusercontent.com/myorg/myrepo/refs/heads/main/funding.yaml
# Any other MyST configuration here
site: ...
project: ...
```

By allowing configuration to be split across multiple files and re-used easily, communities can reduce the duplication and outdated content associated with having a copy of the same information in multiple places. This is particularly useful for communities that need centralized databases of community-wide information (like author information) that they wish to re-use in multiple places. It's also useful to standardize community-specific configuration like branding or links in the site navigation bar.

7.5. Authoring in computational environments with `jupyterlab-myst`

A major motivation for rebuilding Jupyter Book on the MyST ecosystem is to enable a truly **web-native runtime model**. The MyST stack is built in JavaScript, and leverages the `unist` syntax tree ecosystem, a specification for ASTs used in content transformation pipelines, making it directly compatible with modern browser environments and frontend frameworks like React, as well as a host of other plugins and transformations (e.g., `unified-latex`). This also allows us to develop MyST-native authoring applications, such as `jupyterlab-myst`, a plugin that brings MyST rendering capabilities directly into JupyterLab.

With `jupyterlab-myst`, authors can preview how their markdown content elements will appear in the rendered book—including directives, equations, citations, and even code outputs—without leaving the notebook environment. This tight feedback loop reduces friction during authoring, while preserving a single source of truth between exploratory notebooks and published outputs. The plugin is part of our vision to unify **authoring, execution, and publishing** in one environment. Rather than using separate tools for writing, developing, and rendering, the MyST ecosystem will allow interfaces like JupyterLab as a

single full-featured writing and publishing environment. This lowers the barrier to creating high-quality computational narratives, especially for teams already working with Jupyter tools for their day-to-day research and teaching.

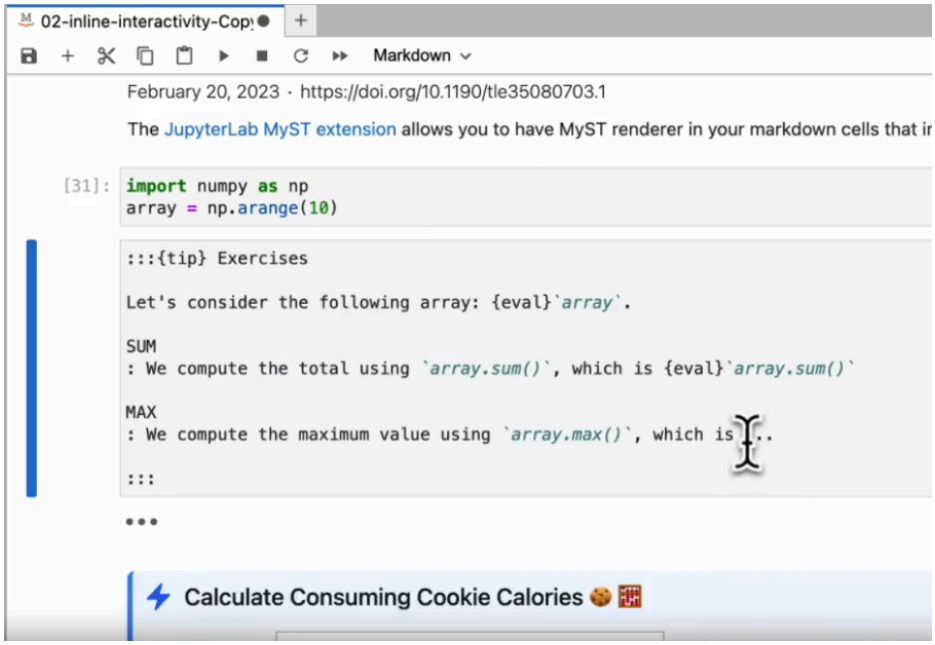


Figure 7. *Using `jupyterlab-myst` to edit MyST content with embedded widgets and interactivity in the markdown cells.*

Acknowledgements

Jupyter Book has been developed over the course of nearly a decade, with contributions from dozens of individuals and organizations. We are grateful to the many collaborators, maintainers, and community members who have helped shape the project's evolution - from its early days as a Jekyll-based tool for compiling notebooks into a textbook, through the Sphinx-based Jupyter Book 1 release, and now with the MyST Markdown and MyST Document Engine-based Jupyter Book 2 architecture. This article did not intend to capture those wide-ranging contributions and ideas or fully document the history of the project, but instead focused on the unique contributions of the Jupyter Book 2 stack.

This project has benefited from key financial support from multiple organizations. The development of MyST Markdown, the Executable Books ecosystem, and Jupyter Book 1 was supported by a grant from the Alfred P. Sloan Foundation (Grant #9231). The development of the MyST Document Engine and the re-architecture of Jupyter Book 2 were made possible in-part by funding from the Alfred P. Sloan Foundation (Grant #9231), Alberta Innovates, The Stanford Doerr School of Sustainability, the Chan Zuckerberg Initiative (2i2c, link), and The Navigation Fund (2i2c, [link] (`https://doi.org/10.71707/r3sy-7371`)). It also included in-kind support from Curvenote, 2i2c, and the broader open-source community. The refactoring of the QIIME 2 Framework documentation ecosystem was supported in part by the NIH National Cancer Institute (Grant 1U24CA248454-01). Project Pythia's transition to Jupyter Book 2 was supported by the National Science Foundation (awards 2324302, 2324303 and 2324304). Notebooks Now! was supported by a grant from the Alfred P. Sloan Foundation (#19361).

We especially acknowledge Chris Sewell, whose foundational work on the MyST Markdown specification, the Sphinx parser (`myst-parser`), and the wider Jupyter Book 1 ecosystem in

Sphinx. Chris's attention to these technical underpinnings laid the groundwork for many of the capabilities now realized in Jupyter Book 2. In addition, we acknowledge John Stachurski for providing key leadership and securing the original funding that created the Jupyter Book 1 ecosystem built on Sphinx.

We also acknowledge the work of the Executable Books team, the contributors to Jupyter Frontends sub-project, Thebe, and the maintainers of MyST-related tooling and plugins. The ongoing development of Jupyter Book 2 is supported and stewarded by the Jupyter Book Community, and is now a subproject of Project Jupyter.

Authors are in alphabetical order after "Project Jupyter" following Project Jupyter's approved process for authoring Jupyter related academic papers and aligned with our shared values of inclusivity and generosity in authorship, following clear, explicit criteria for authorship, openness, and personal accountability.

References

[1] Executable Books Community, "Jupyter Book." [Online]. Available: https://zenodo.org/record/2561065

[2] B. E. Granger and F. Perez, "Jupyter: Thinking and Storytelling With Code and Data," *Computing in Science & Engineering*, vol. 23, no. 2, pp. 7–14, 2021, doi: 10.1109/mcse.2021.3059263.

[3] K. Thomas *et al.*, "Jupyter Notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, IOS Press, 2016. doi: 10.3233/978-1-61499-649-1-87.

[4] R. Cockett, S. Purves, F. Koch, and M. Morrison, "Continuous Tools for Scientific Publishing: Using MyST Markdown and Curvenote to encourage continuous science practices," in *Proceedings of the 23rd Python in Science Conference*, in SciPy. 2024, pp. 121–136. doi: 10.25080/nkvc9349.

[5] Fernando Pérez, "AGU Notebooks Now! Evaluation.." [Online]. Available: https://zenodo.org/doi/10.5281/zenodo.15061830

[6] R. Cockett, L. J. Heagy, and D. W. Oldenburg, "Pixels and their neighbors: Finite volume," *The Leading Edge*, vol. 35, no. 8, pp. 703–706, 2016, doi: 10.1190/tle35080703.1.

[7] R. Cockett, C. Babott, and S. Purves, "Continuous Science Foundation Workshop — Banff 2025: Shaping the future of scientific communication through Stories, Values, and Movement from Banff," 2025, doi: 10.62329/hytv4378.

[8] P. Jupyter *et al.*, "Binder 2.0 - Reproducible, interactive, sharable environments for science at scale," in *Proceedings of the 17th Python in Science Conference*, in SciPy. 2018, pp. 113–120. doi: 10.25080/majora-4af1f417-011.

[9] The Turing Way Community, "The Turing Way: A handbook for reproducible, ethical and collaborative research." [Online]. Available: https://zenodo.org/doi/10.5281/zenodo.3233853

[10] B. E. J. Rose *et al.*, "Collaborative Research: GEO OSE TRACK 2: Project Pythia and Pangeo: Building an inclusive geoscience community through accessible, reusable, and reproducible workflows," 2023, doi: 10.5281/ZENODO.8184298.